

Chapter 5 · Lesson 3 — Building PlayerDetail (Case Study)

Chapter 5 · Lesson 3 — Building PlayerDetail (Case Study)

Goal: study a real, deeply nested feature view — the Players tab and its <PlayerDetail> drill-down — to see composition, prop drilling, conditional rendering, and back-and-forth navigation in action.

The problem

The user clicks a player card from a 32-player grid. They land on a deep “player detail” view: the player’s tournament path (R1 → R2 → QF → SF → F won/lost/live), four stat tiles, and a per-match breakdown showing which fantasy teams backed them and earned what points. The opponent’s name in each match analysis is **clickable** — clicking it navigates to *that* player’s detail view.

Now hit “Back” and you’re returned to the grid.

The component tree

```
<PlayersTab teams>
├─ if no player selected:
│   └─ grid of <PlayerCard /> (32 of them)
└─ if a player IS selected:
    └─ <PlayerDetail playerName teams onBack onSelectPlayer>
        ├── header (gradient, name, status, path bar)
        ├── grid of <StatTile /> (4 of them)
        └─ for each match the player appeared in:
            └─ <MatchPickAnalysis analysis playerName onSelectPlayer>
```

```

└─ header (round, players, outcome)
└─ BACKED <playerName> column
    └─ list of <TeamChip team ptsEarned />
└─ BACKED <opponent> column
    └─ list of <TeamChip team ptsEarned />

```

Six components, four levels of nesting. **None of it more than ~70 lines.** Each component has one job.

Step 1: PlayersTab and the conditional render

```

'use client';

import { useState, useMemo } from 'react';
import type { TeamWithScores } from '@/lib/types';
import { PLAYER_INFO } from '@/lib/players';
import { QF_MATCHES, SF_MATCHES, FINAL_MATCH } from '@/lib/matches';
import PlayerCard from '../players/PlayerCard';
import PlayerDetail from '../players/PlayerDetail';

interface PlayersTabProps {
  teams: TeamWithScores[];
}

export default function PlayersTab({ teams }: PlayersTabProps) {
  const [selectedPlayer, setSelectedPlayer] = useState<string | null>(null);

  // ... (build allPlayers list, compute who's still in)

  if (selectedPlayer) {
    return (
      <PlayerDetail
        playerName={selectedPlayer}
        teams={teams}
        onBack={() => setSelectedPlayer(null)}
        onSelectPlayer={setSelectedPlayer}

```

```

        />
    );
}

// ... else render the grid
return (
    <div>
        {filtered.map(p => (
            <PlayerCard key={p.name} player={p} onClick={() => setSelectedPlayer
        ))}
    </div>
);
}

```

Two key ideas here.

Local state for the detail view

```
const [selectedPlayer, setSelectedPlayer] = useState<string | null>(null);
```

`selectedPlayer` is local to `<PlayersTab>`. **No other tab cares which player is open.** So it doesn't need to live at the orchestrator level.

This is the **lowest common ancestor** rule from Chapter 4 in action: the readers (`PlayersTab`, `PlayerDetail`, `MatchPickAnalysis`) and writers (`PlayerCard` click, `opponent name` click) all live inside the `PlayersTab` subtree. So state lives there.

Compare with `selectedTeam` from Chapter 4 — that lives at the orchestrator because writers (`Standings` click) and readers (`Predictions` tab) span different sub-trees.

Selected by name, not by object

```

useState<string | null>(null);
// ...
onSelectPlayer={setSelectedPlayer}
// passes a `string` (player name)

```

We store **the player's name** (a string), not the full player object. Why?

- **Stability across re-renders.** A string is primitive; React can compare it cheaply. An object reference would change if we ever re-derived the players list.
- **Simplicity.** The detail view can look up its own player metadata from `PLAYER_INFO[playerName]` and the matches the player appeared in via `.find()` calls. No need to pass the whole object around.
- **Matches the data shape.** `PLAYER_INFO` is keyed by name, so name-as-id is natural.

The trade-off: the detail view does a few `.find()` calls to resolve the name into rich data. Microseconds. Worth the simplicity.

The conditional return

```
if (selectedPlayer) {  
  return <PlayerDetail ... />;  
}  
return <div>{/* grid */</div>;
```

Two completely different views, gated on whether `selectedPlayer` is set. **Early return** is a clean way to handle “two-mode” components.

The alternative is a single JSX with `{selectedPlayer ? <PlayerDetail /> : <div>{grid}</div>}`. Both work; early return is more readable when the two branches are very different. Use early return when the difference is structural (different layout entirely); use ternary when the difference is a single element.

Step 2: PlayerCard – the leaf

`<PlayerCard>` is a 30-line button. The whole job: render one player's name, seed, country, eliminated/alive status. Click handler.

```
interface PlayerCardProps {  
  player: { name: string; seed?: number; status: string; flag: string; still  
  onClick: () => void;  
  finalPicksCount?: number;  
}
```

```
export default function PlayerCard({ player, onClick, finalPicksCount }: PlayerCardProps) {
  return (
    <button onClick={onClick} style={{ /* ... */ }}>
      {/* seed badge */}
      {/* champion / runner-up / out badge */}
      {/* name */}
      {/* country flag */}
      {/* hint chip with finalPicksCount if set */}
    </button>
  );
}
```

Three things to note:

A button, not a div

The card is a `<button>`. Why? **Accessibility**. Screen readers announce buttons differently from divs; keyboard users can Tab to a button and Enter on it. A `<div onClick>` traps both.

This is one of those small senior moves that interviewers love to spot. *“This div should be a button.”* — if you can articulate that distinction, you’ve leveled up.

Optional finalPicksCount prop

```
finalPicksCount?: number;
```

When set, the card shows a small chip like *“+3 each from 6 teams”*. When undefined, no chip. The component handles both cases with `{finalPicksCount !== undefined && <chip />}`.

This is the **optional-feature-via-optional-prop** pattern. The card stays simple in the no-chip case; callers can opt in by passing the prop.

Hover effects via inline onMouseOver / onMouseOut

```
onMouseOver={(e) => { e.currentTarget.style.transform = 'translateY(-3px)'; }}
onMouseOut={(e) => { e.currentTarget.style.transform = 'none'; }}
```

This is the inline-styles equivalent of a CSS `:hover`. Works fine; slightly verbose. If you had 100 cards needing the same hover, you'd extract it into CSS. For 32 cards on one screen, inline is acceptable.

A more "CSS-purist" version would use CSS Modules or Tailwind's `hover:` modifier. The codebase chose inline because every card already has dynamic colors that don't fit CSS classes. **Consistency within a component matters more than purity.**

Step 3: PlayerDetail – the big one

`<PlayerDetail>` is ~270 lines. Let's not paste it all – open `components/players/PlayerDetail.` to follow along. Instead, let's see its **structure**:

```
export default function PlayerDetail({ playerName, teams, onBack, onSelectPl
  // 1. Look up player metadata
  const info = PLAYER_INFO[playerName] || {};

  // 2. Find which matches the player appeared in (across all 5 rounds)
  const r1Idx = ROUND1_MATCHES.findIndex(m => m.p1 === playerName || m.p2 ===
  const r2Idx = ROUND2_MATCHES.findIndex(m => m.p1 === playerName || m.p2 ===
  // ...

  // 3. Build a `pickAnalyses` array – one entry per match-the-player-was-in
  const pickAnalyses: PickAnalysis[] = matches.map(mp => {
    const opponent = mp.match.p1 === playerName ? mp.match.p2 : mp.match.p1;
    const teamsBackedThis: TeamWithScores[] = [];
    const teamsBackedOpponent: TeamWithScores[] = [];
    teams.forEach(t => {
      const pick = mp.pickKey === 'final' ? t.final : t[mp.pickKey]?.[mp.mat
      if (pick === playerName) teamsBackedThis.push(t);
      else if (pick === opponent) teamsBackedOpponent.push(t);
    });
    return { ...mp, opponent, teamsBackedThis, teamsBackedOpponent };
  });

  // 4. Compute aggregate stats
```

```

const totalPicks = pickAnalyses.reduce((s, m) => s + m.teamsBackedThis.length, 0)
// ...

// 5. Render header, stats, match-by-match analyses
return (
  <div>
    <button onClick={onBack}><- Back to All Players</button>
    <Header playerName={playerName} info={info} stillIn={...} />
    <StatTilesRow stats={...} />
    {pickAnalyses.map((pa, i) => (
      <MatchPickAnalysis key={i} analysis={pa} playerName={playerName} onS
    ))}
  </div>
);
}

```

Five phases: lookup, find matches, build analyses, compute aggregates, render. Each phase is a few lines. The whole thing is large because there are many phases, not because any single phase is complex.

Why doesn't this need useMemo?

You might think *“this component is computing a lot – should it be memoized?”*

The answer: **no**, because the computation runs once per render of `<PlayerDetail>`, and `<PlayerDetail>` only re-renders when its parent (`<PlayersTab>`) re-renders, which only happens when `selectedPlayer` changes. Each new player triggers a new computation; that's correct. There's no waste.

If `<PlayersTab>` re-rendered for unrelated reasons (e.g. a parent state change), then `<PlayerDetail>` would recompute too – wasteful. We could `React.memo` it then. For this app's size, that's premature.

The `pickAnalyses.map` is the most important transform

This is the data shape the inner component (`<MatchPickAnalysis>`) needs:

```
interface PickAnalysis {
  round: string;
  roundFull: string;
  bestOf: string;
  matchIndex: number;
  match: Match;
  finished: boolean;
  pickKey: 'r1' | 'r2' | 'qf' | 'sf' | 'final';
  opponent: string;
  won: boolean | null;
  teamsBackedThis: TeamWithScores[];
  teamsBackedOpponent: TeamWithScores[];
}
```

Each entry is one match the player was in, decorated with everything the inner component will display. **All the heavy lifting happens in the parent**; the inner component is pure presentation.

This is a key composition pattern: **the parent does the data shaping; the child renders the shape**. Don't make children do their own data lookups — pass them the ready-to-render data.

Step 4: MatchPickAnalysis — the per-match section

About 150 lines. Receives one PickAnalysis and renders:

```
<div>
  {/* round badge + matchup title + outcome chip */}
  <Header />

  {/* "X of 8 teams backed Wu Yize · Y of 8 backed Murphy" */}
  <Summary />

  {/* two columns of TeamChips */}
  <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(150px, 1fr))' }}>
    <BackedColumn label={`BACKED ${playerName}`} teams={analysis.teamsBackedThis} />
    <BackedColumn label={`BACKED ${analysis.opponent}`} teams={analysis.teamsBackedOpponent} />
  </div>
</div>
```



```
</div>
</div>
```

(Note `<BackedColumn>` is **inlined** here, not extracted — covered in Lesson 1’s “when not to extract” section.)

The clickable opponent name

The most interesting piece:

```
<button
  onClick={() => onSelectPlayer(analysis.opponent)}
  style={{ background: 'none', border: 'none', textDecoration: 'underline',
>
  {analysis.opponent}
</button>
```

Clicking the opponent name calls `onSelectPlayer(opponentName)` — the **same setter** that originally got us into `<PlayerDetail>`. This effectively:

1. Sets `selectedPlayer` in `<PlayersTab>` to the opponent’s name.
2. Triggers a re-render of `<PlayersTab>`.
3. `<PlayerDetail>` re-renders with the new `playerName` prop.
4. The whole detail view recomputes for the new player.

That’s **navigation by state change**, not by URL or routing. We jump from one detail view to another by changing one piece of state. The user sees a smooth transition; under the hood it’s React reconciling the new tree.

The callback chain

`onSelectPlayer` was passed:

- From `<PlayersTab>` to `<PlayerDetail>`.
- From `<PlayerDetail>` to `<MatchPickAnalysis>`.

Two levels of drilling. Lesson 2’s pain threshold is 4–5 levels. Two is fine.

If we ever moved player navigation to a global URL-based system (e.g. `/players/wu-yize`), this drilling would disappear and the navigation would become a `<Link>` from

next/link. **The drilling exists because the navigation is in-memory state.** Different design, different prop flow.

Step 5: TeamChip and StatTile — the leaves

We've already seen `<TeamChip>` from Lesson 1. `<StatTile>` is similar:

```
interface StatTileProps {
  label: string;
  value: string | number;
  sub?: string;
  icon: string;    // emoji
}

export default function StatTile({ label, value, sub, icon }: StatTileProps)
  return (
    <div style={{ /* card background */ }}>
      <div style={{ fontSize: 28 }}>{icon}</div>
      <div style={{ /* small label */ }}>{label}</div>
      <div style={{ /* big value */ }}>{value}</div>
      {sub && <div style={{ /* small sub */ }}>{sub}</div>}
    </div>
  );
}
```

Four props, ~20 lines, no logic. Pure presentation. Used 4 times in `<PlayerDetail>`.

These tiny presentation components are **the bedrock of a reusable codebase**. Each one is testable in isolation, swappable, and easy to refactor. The complexity lives at the orchestrator level; the leaves stay simple.

Step 6: PathStep and PathArrow — the readability components

We talked about these in Lesson 1. They exist purely to make the parent's path JSX read like a sentence:

```
<PathStep label="R1" status="won" />
<PathArrow />
```

```

<PathStep label="R2" status="won" />
<PathArrow />
<PathStep label="QF" status="lost" />

```

<PathStep> maps a 4-state union ('won' | 'lost' | 'live' | 'na') to four different visual treatments. The map lives inside <PathStep>:

```

const styles: Record<PathStatus, { bg, color, border, icon }> = {
  won: { bg: 'rgba(255,255,255,0.95)', color: '#0F5132', icon: '🏆' },
  lost: { bg: 'rgba(220, 38, 38, 0.85)', color: '#FFF', icon: '🏆' },
  live: { bg: '#FBBF24', color: '#0F5132', icon: '●' },
  na: { bg: 'rgba(0,0,0,0.18)', color: 'rgba(255,255,255,0.55)', icon: '—' }
};

```

A **lookup object** of styles, indexed by the status string. Cleaner than a chain of ternaries. **Use lookup objects whenever you have 3+ branches mapping a value to a style/config.**

What we just learned about composition

Read back through what <PlayerDetail> and its descendants accomplish:

- **270 lines of orchestration** (<PlayerDetail>).
- **Six leaf components**, each 15–60 lines (<PlayerCard>, <MatchPickAnalysis>, <TeamChip>, <StatTile>, <PathStep>, <PathArrow>).
- **No useMemo, no Context, no global state.** Just useState at the right level + prop drilling for two pieces of data (teams, onSelectPlayer).
- **Self-referential navigation** — clicking the opponent name in a detail view loads *their* detail view via the same state setter.

That's a fully-functional 32-player drill-down feature, built with **only the techniques in Chapters 3 and 4**. No new React concepts in this chapter — just **the discipline to apply them well**.

Vibe prompts for the Players tab

Building it from scratch would have been three or four prompts:

1. *"Create <PlayersTab> at components/tabs/PlayersTab.tsx. Top-level filter buttons: Still In, Eliminated, All. Render a grid of <Play-*

erCard> for each player from `PLAYER_INFO`. Compute `stillIn` from `QF_MATCHES + SF_MATCHES + FINAL_MATCH` (a player is still in if they appear in QF and haven't lost since). Mark whoever won the Final as champion. State: `selectedPlayer: string | null`. Conditional render `<PlayerDetail>` if set."

2. "Create `<PlayerCard>` at `components/players/PlayerCard.tsx`. Button. Props: `player: PlayerCardData`, `onClick: () => void`, `finalPicksCount?: number`. Visuals: seed badge, status badge (champion/runner-up/out), name, country flag, optional chip showing how many teams picked them in the Final."
3. "Create `<PlayerDetail>` at `components/players/PlayerDetail.tsx`. Props: `playerName`, `teams`, `onBack`, `onSelectPlayer`. Look up info from `PLAYER_INFO`, find every match the player appeared in, build a `pickAnalyses` array decorating each match with which teams backed the player vs the opponent. Render header (gradient, name, status, tournament-path bar), 4 stat tiles, then map `pickAnalyses` through `<MatchPickAnalysis>`."
4. "Create `<MatchPickAnalysis>` at `components/players/MatchPickAnalysis.tsx`. Props: `analysis: PickAnalysis`, `playerName`, `onSelectPlayer`. Render the matchup header (round, players, outcome with score). Make the opponent's name a button calling `onSelectPlayer(opponent)`. Below: two columns of `<TeamChip>`s, one for each side. Color the points chip green for +3, brown for +1."

Each prompt is **one component, one paragraph, names every prop**. That's the right grain. Compose by prompts the same way you compose by components.

CHECK YOURSELF

1. `selectedPlayer` lives in `<PlayersTab>`. Why not in `<SnookerFantasyLeague>`?
2. The opponent's name in `<MatchPickAnalysis>` is a `<button>` calling `onSelectPlayer(opponent)`. What state changes? In which component? How many re-renders happen?
3. `<PlayerDetail>` does 5 phases of computation in its function body before return-

ing JSX. Why isn't it wrapped in `useMemo`? What would have to be true to memoize it?

4. `<PathStep>` uses a lookup object `{ won: ..., lost: ..., live: ..., na: ... }` instead of `if/else if` chains. Why is this better?
5. Open `components/players/PlayerDetail.tsx`. Find the `pickAnalyses.map(...)` block. Walk through what it produces. Specifically: for each match the player appeared in, what does each `PickAnalysis` entry look like, and where does it get used downstream?

When you've answered these, you've finished Chapter 5. Onward to **Chapter 6 – Data Visualization** – the analytics tab and Recharts.